8-3 · 수집→Lake→Spark→VectorDB 흐름 자동화 · 실습 1H

임베딩→VectorDB 적재와 전체 DAG 완성

투입구가 정본, 지식베이스는 그 그림자

수도권ICT 이노베이션스퀘어

데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① 두 번 돌려도 lecture_kb 가 11 그대로인 것(멱등)을 확인한다

② 투입구에서 지운 문서의 조각이 남는 구멍을 실측한다

③ prune 태스크를 더해 다섯 태스크로 완성한다 — 11 → 10 → 11

④ @daily 를 붙이고 SparkSubmitOperator 의 DAG 모양을 비교한다

1.5H 의 rag_pipeline(네 태스크 · lecture_kb 11)에서 이어 갑니다

실습 흐름

여섯 단계 — 멱등에서 실행 이력까지

2단계에서 구멍을 보고, 3단계에서 메웁니다

1. 두 번 돌리기	2. 문서 지우기	3. prune	4. 스케줄	5·6. Spark·이력
count 11 그대로	success 인데 11 그대로	11 → 10 되돌리면 11	@daily next-execution	구조 확인 list-runs

1단계

두 번 돌려 멱등을 확인하세요

1. **lecture_kb points/count → 11**
   지금 개수를 먼저 본다
2. **af dags trigger rag_pipeline → sleep 60**
   한 번 더 끝까지 돌린다
3. **다시 세면 11 그대로**
   build_kb.py 는 문서마다 옛 조각을 지우고 다시 넣는다

조각 id

출처#순번

(7-4)

2단계 — 일부러 구멍 보기

투입구에서 문서를 지워 보세요

1. **mv dags/data/incoming/airflow_ops.txt ~/**
   잠시 밖으로 — 그리고 trigger · sleep 60

2. **네 태스크 success 인데 count 는 11 그대로**
   scroll 로 보면 1 incoming/airflow_ops.txt 가 남아 있다(실측)

3. **collect 는 MinIO 에서 지웠다**
   build_kb.py 는 버킷에 있는 문서만 다시 넣을 뿐 — 사라진 조각은 그대로

4. **챗봇은 지운 문서로 계속 답한다**
   실측 — 지운 문서의 질문이 0.517 로 1등

7-4 의

줄어든 문서

같은 구멍

3단계

prune — 버킷에 없는 조각을 지운다

live 목록	before	delete	출력
버킷의 .txt 를 minio://... 출처로	qc.count (COLLECTION)	Filter(must_not=... MatchAny(any=live))	정리: before → after 조각

source 가 지금 있는 문서 목록에 없는 조각만 지운다 — 파일 전체를 cat > 로 다시 쓰고 sleep 35

build_index 뒤 · report 앞 — t_collect >> t_validate >> t_index >> t_prune >> t_report

3단계 · 결과

다섯 태스크를 돌려 보세요 — 11 → 10 → 11

1. **trigger → states-for-dag-run**
   collect · validate · build_index · prune · report 다섯 줄 success

2. **count 10 — prune Logs 에 정리: 11 → 10 조각**
   지식베이스가 7-5 끝 모양으로 돌아왔다(실측)

3. **mv ~/airflow_ops.txt dags/data/incoming/ → trigger**
   count 가 다시 11 — 8-4 챗봇이 이 11 조각으로 답한다

정본

투입구

4단계

스케줄을 붙이세요 — 사람이 손대지 않아도 매일

1. **python3 - << 'EOF' 로 schedule=None → "@daily"**
   grep -n schedule 에 schedule="@daily", 가 보인다 · sleep 35
2. **af dags next-execution rag_pipeline**
   다음 실행 시각 한 줄 — 오늘 자정 UTC(한국 오전 9시)
3. **schedule=None 일 때는**
   No following schedule can be found (실측)
4. **catchup=False 를 함께 둔다**
   start_date(2026-01-01)부터 밀린 날짜가 쏟아지지 않는다

이번 주 목표

손대지 않아도
매일 돈다

5단계

Spark 제출 — 태스크 안의 실행 방식만 다르다

오늘 — PythonOperator

build_kb.py 를 부른다, 단일 노트북에서 돈다

- task_id="build_index"
- python_callable=build_index
- t_validate >> t_index >> t_prune

VS

실무 — SparkSubmitOperator

Spark 클러스터에 잡을 제출한다(실행하지 않음)

- application=/opt/airflow/jobs/transform.py
- conn_id="spark_default"
- t_validate >> [transform_spark, t_index]

6단계

실행 이력을 살펴보세요

1. **af dags list-runs rag_pipeline -o plain | head -8**
   오늘 트리거한 실행이 한 줄씩 — success 와 failed(검증 실패)

2. **한 실행의 태스크별 상태 · 시각**
   지금까지 쓴 af tasks states-for-dag-run

3. **오래 걸리는 태스크 = 최적화할 곳**
   오늘은 build_index

오케스트레이션의 값

기록이 남는다

확인 포인트

이 1시간이 끝날 때 확인할 것

1. 두 번 돌려도 lecture_kb 가 11 그대로다
2. 문서를 지우면 prune 없이는 조각이 남는 것을 봤다
3. 다섯 태스크 success · prune 뒤 10 · 되돌리면 11
4. next-execution 이 다음 실행 시각을 한 줄로 낸다
5. list-runs 에 오늘 실행이 success · failed 로 남아 있다

Spark 를 오늘 돌리지 않는 이유는 메모리 — 배선은 같다

8-3 · 실습 1H 마무리

다음은 정리·회고 (0.5H)

오늘 쓴 방법을 표로 정리하고, 8-4 챗봇이 읽을 lecture_kb 가 11 조각인지 확인합니다

- ✓ 하려는 일 ↔ 방법 요약
- ✓ 체크리스트로 스스로 점검
- ✓ 자주 겪는 오류 네 가지
- ✓ 8-4 Streamlit 챗봇 예고

가져갈 것

lecture_kb 11 · rag_pipeline 다섯 태스크